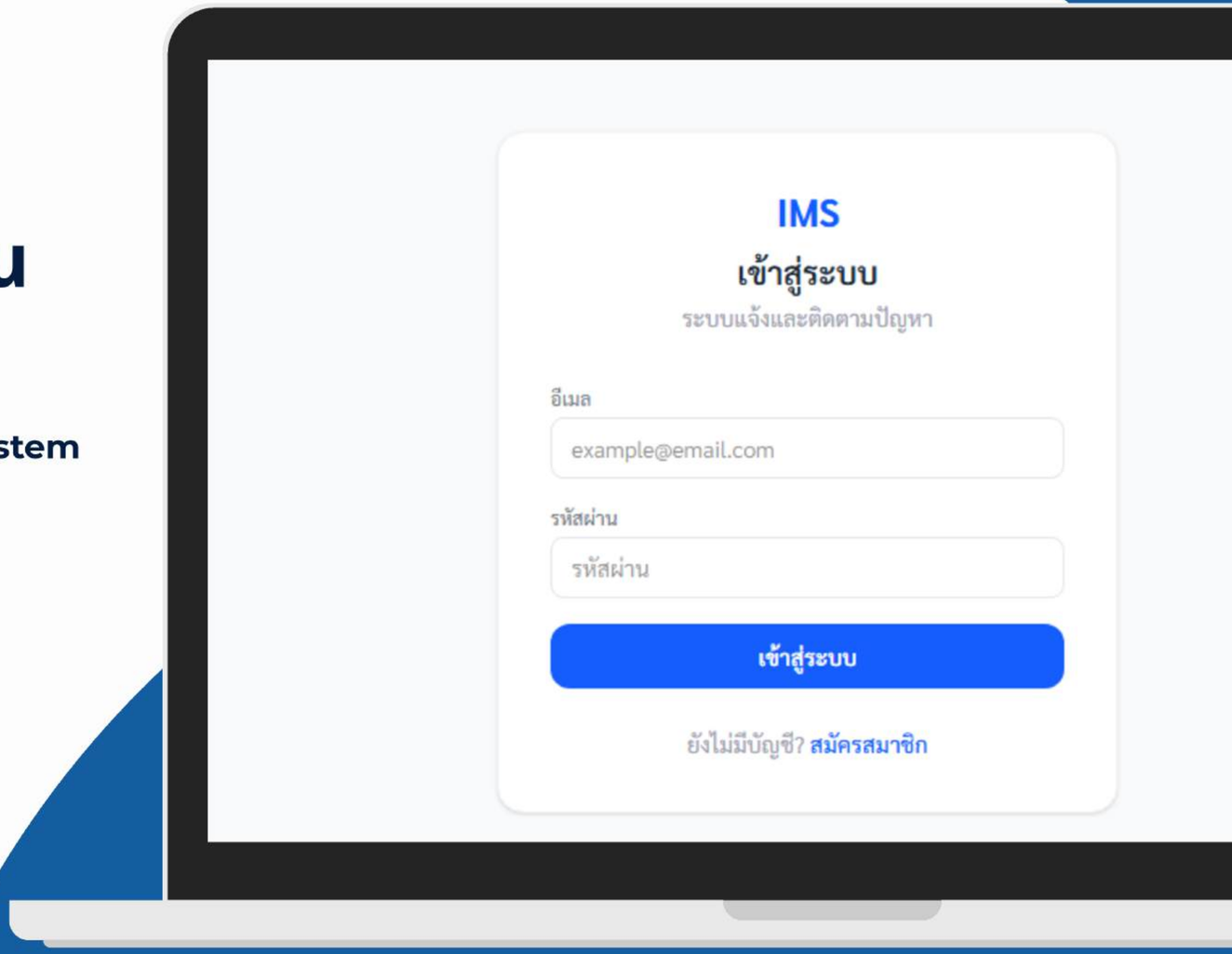
ระบบร้องเรียนปัญหาและแจ้งเตือนข่าวสารภายในมหาวิทยาลัย

University Issue Reporting and Emergency Notification System



65160116 นายนนทวัฒน์ สุขยิ่ง
6516038 นางสาวบุศรินทร์ พันธุเวช
65160385 นางสาวพิชญาดา เกิดสุทธิ
65160246 นางสาวณัฏฐกาญน์ ผึ่งกรรฐ์
65160379 นางสาวนิรชา แก้วงาม
65160396 นางสาวอารี แก้วสีคร้าม

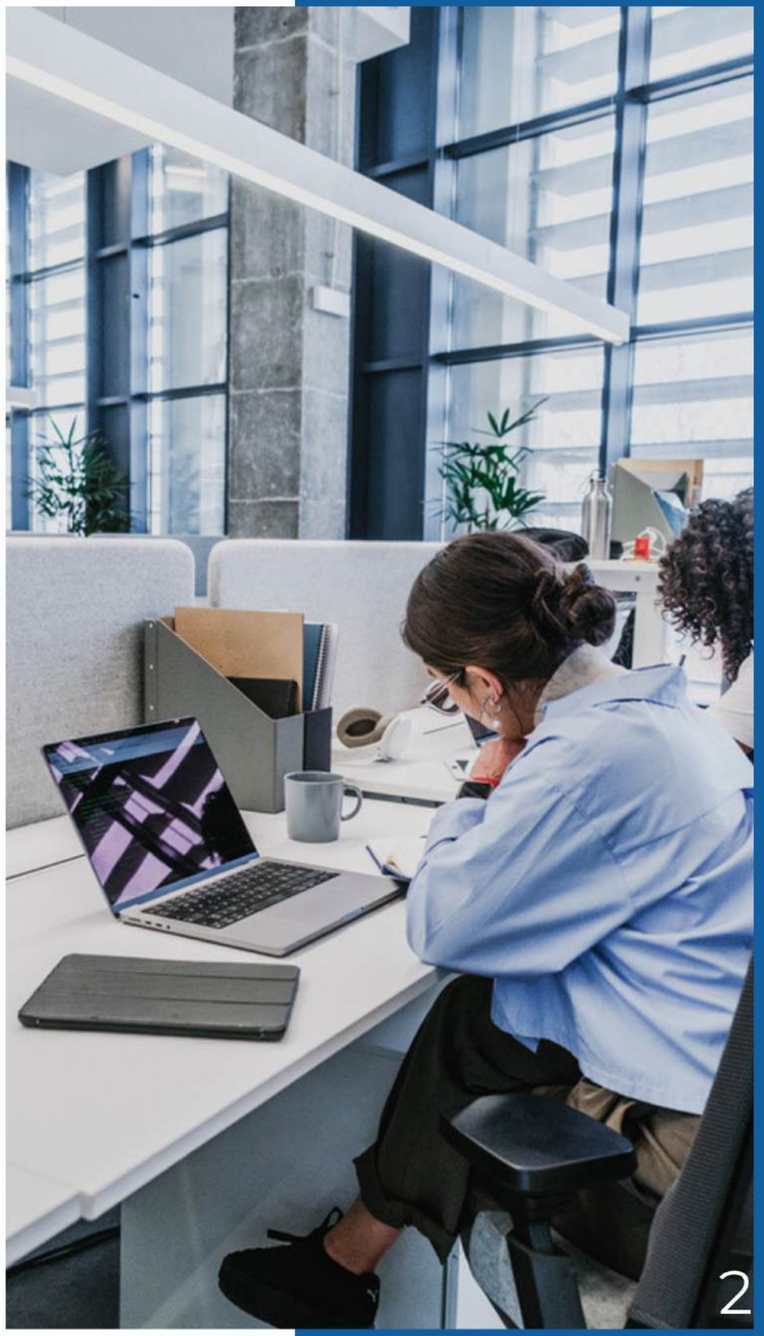
Group 6

ระบบร้องเรียนปัญหาและ แจ้งเตือนข่าวสารภายในมหาวิทยาลัย คืออะไร?

ระบบนี้สร้างขึ้นเพื่อใช้ในการแจ้งปัญหาและสื่อสารข่าวสารภายในมหาวิทยาลัย โดยเปิดโอกาสให้นิสิตและบุคลากรสามารถรายงานปัญหาได้อย่างสะดวก รวมถึงช่วยให้การสื่อสารระหว่างผู้ใช้งานและหน่วยงานที่เกี่ยวข้องมีประสิทธิภาพมากขึ้น และทำให้การจัดการปัญหาภายในมหาวิทยาลัยมีความเป็นระบบมากยิ่งขึ้น

ความสำคัญของระบบ

ช่วยลดความล่าช้าในการแจ้งและแก้ไขปัญหาภายในมหาวิทยาลัย เพิ่มประสิทธิภาพในการสื่อสารและการประสานงานระหว่างหน่วยงาน ทำให้ข้อมูลสามารถตรวจสอบและติดตามได้อย่างเป็นระบบ



ขอบเขตระบบ

ระบบร้องเรียนปัญหาและแจ้งเตือนข่าวสารภายในมหาวิทยาลัย พัฒนาขึ้นเพื่อเป็นสื่อกลางในการบริหารจัดการปัญหาและกระจายข้อมูลอย่างมีประสิทธิภาพ โดยมีขอบเขตดังนี้

1. **ระบบจัดการสิทธิ์**
   - เข้าใช้งานด้วย อีเมลหน่วยงานเท่านั้น
   - แบ่งสิทธิ์ตามบทบาท 5 กลุ่มผู้ใช้งาน

2. **ระบบแจ้งเรื่องร้องเรียน**
   - ระบุสถานที่ด้วย Dropdown หรือระบุเองในกรณีอื่นๆ
   - รองรับการแนบหลักฐานรูปภาพ

3. **ระบบบริหารจัดการงาน**
   - คัดกรองหมวดหมู่และกำหนดระดับความเร่งด่วน
   - ส่งต่องานให้หน่วยงานที่เกี่ยวข้องได้โดยตรง

4. **ระบบติดตามและประเมินผล**
   - ติดตามไทม์ไลน์สถานะการดำเนินงานได้ตลอดเวลา
   - ระบบประเมินความพึงพอใจหลังจบงาน

เป้าหมาย

สร้างแพลตฟอร์มส่วนกลาง

เพื่อเป็นศูนย์กลางทางดิจิทัลที่เชื่อมโยงผู้ใช้งานเข้าไว้ด้วยกัน

เพิ่มประสิทธิภาพการทำงาน

เพื่อให้กระบวนการรับแจ้ง ตรวจสอบ คัดกรอง และแก้ไขปัญหาภายในมหาวิทยาลัยเป็นไปอย่างรวดเร็ว เป็นระบบ และตรวจสอบได้

ยกระดับการสื่อสาร

เพื่อให้การแจ้งเตือนข่าวสารและเหตุฉุกเฉินต่างๆ ส่งถึงผู้ที่เกี่ยวข้องแบบเรียลไทม์

ปัญหา

แจ้งข้อมูลแล้วหาย

ผู้ใช้งาน ทั้งนิสิตและบุคลากร เมื่อทำการแจ้งปัญหาต่างๆ เข้ามาแล้ว กลับไม่สามารถติดตามความคืบหน้าได้

ข้อมูลไม่ครบถ้วน

ข้อมูลที่ได้รับแจ้งมักมีความคลาดเคลื่อน เช่น การระบุพิกัดสถานที่ผิดพลาด หรือขาดภาพถ่ายหลักฐานประกอบ ส่งผลให้การเข้าแก้ไขปัญหาหน้างานเกิดความล่าช้า

แจ้งเหตุฉุกเฉินไม่ได้

ขาดช่องทางที่สามารถกระจายข่าวสารสำคัญหรือแจ้งเตือนเหตุฉุกเฉินให้บุคลากรรับทราบได้อย่างรวดเร็ว

สิ่งที่เราทำ

ระบบติดตามงานเรียลไทม์

สร้างระบบที่มี Dashboard ติดตามสถานะ (Real-time Tracking) ผู้แจ้งสามารถเข้ามาดูไทม์ไลน์ได้ตลอดเวลาว่าคำร้องอยู่ในขั้นตอนไหน

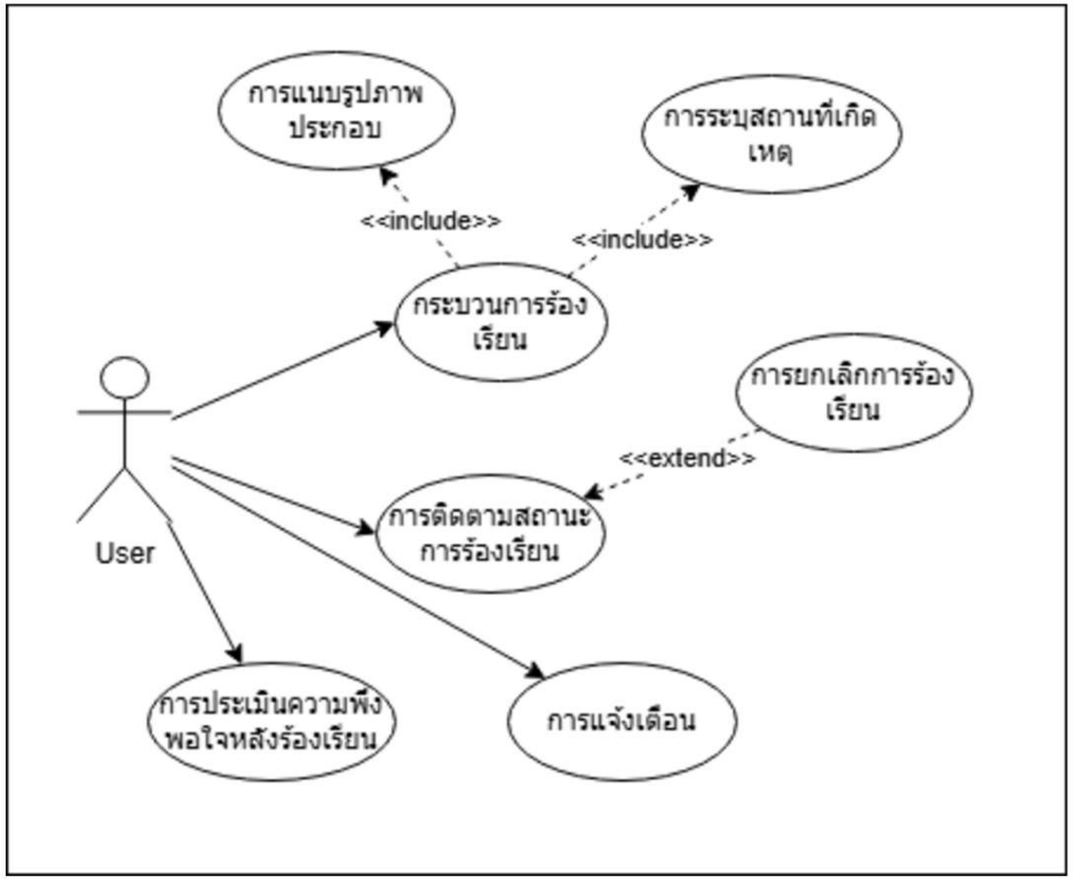
ระบบกรอกข้อมูลพิกัดแม่นยำ

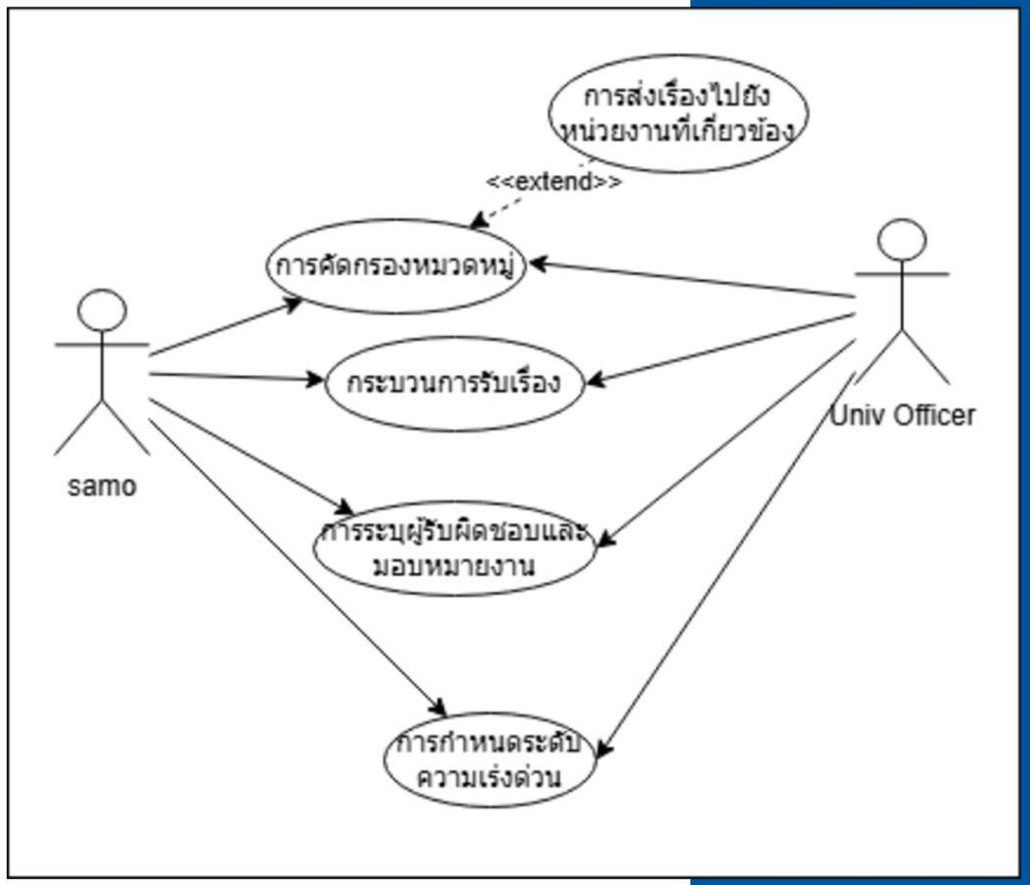
ปรับเปลี่ยนรูปแบบการกรอกข้อมูล โดยใช้ ระบบ Dropdown List ดึงข้อมูลอาคารและสถานที่จากฐานข้อมูลกลางของมหาวิทยาลัยมาให้เลือก และมีการแนบรูปภาพ

การแจ้งเตือนฉุกเฉิน

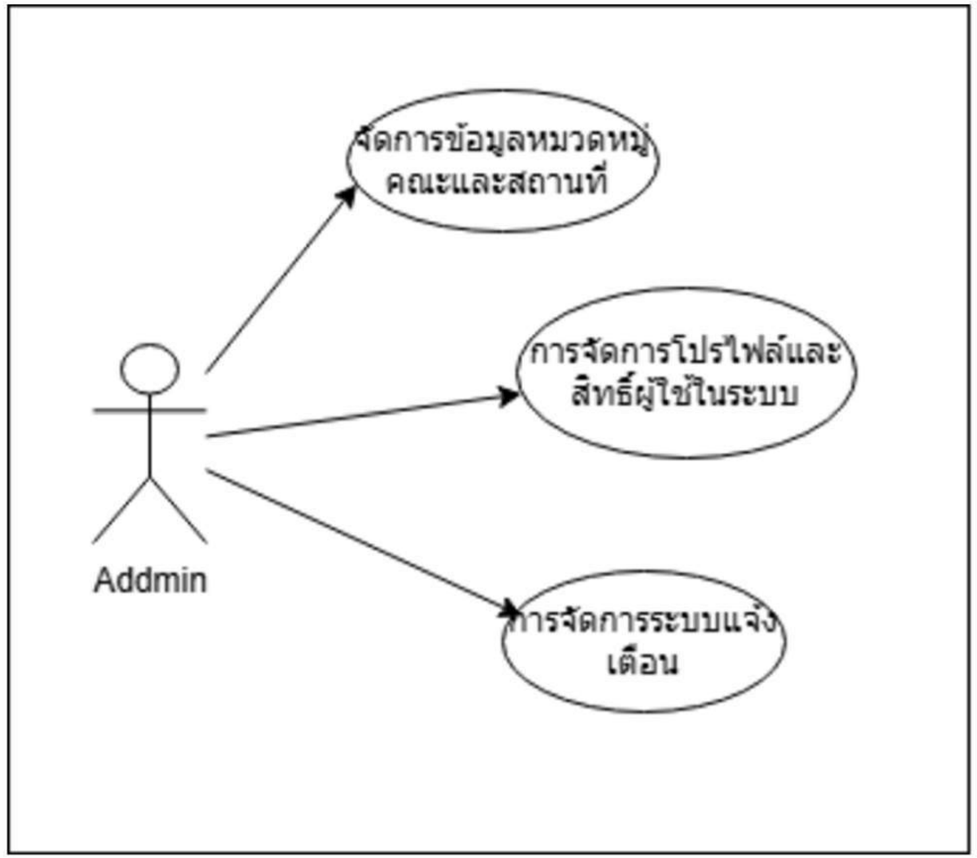
พัฒนาฟังก์ชัน ประกาศแจ้งเตือนจากระบบ ที่ให้ผู้ดูแลระบบสามารถพิมพ์ข้อความและส่งแจ้งเตือนข่าวสารสำคัญหรือเหตุฉุกเฉิน ตรงไปยังหน้า Dashboard และอีเมลของผู้ใช้งานทุกคน

Use Case Diagram





Use Case Diagram



จุดเด่น

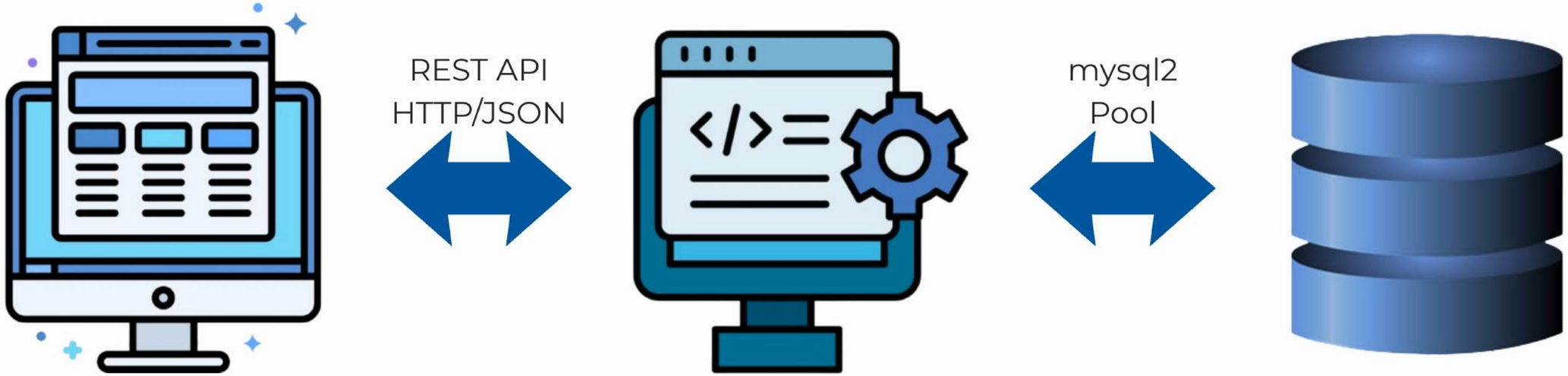
- **การระบุสถานที่อย่างแม่นยำ** ระบบกำหนดให้ผู้ใช้งานระบุสถานที่ผ่านรายการตัวเลือก เพื่อป้องกันความคลาดเคลื่อนและลดข้อผิดพลาดจากการกรอกข้อมูล

- **การติดตามสถานะแบบเรียลไทม์** ผู้ร้องเรียนสามารถติดตามความคืบหน้าของคำร้องผ่านระบบได้ตลอดเวลา โดยแสดงสถานะอย่างเป็นลำดับขั้นตอน

- **การควบคุมการเข้าถึงตามบทบาท** ระบบมีการจัดสรรสิทธิ์และปรับเปลี่ยนอินเทอร์เฟซการใช้งานให้สอดคล้องกับประเภทของผู้เข้าสู่ระบบ เพื่อให้การบริหารจัดการข้อมูลของแต่ละกลุ่ม

โครงสร้างระบบ

โครงสร้างใช้ 3-Layer MVC



Frontend
View Layer

ทำหน้าที่แสดงผลอย่างเดียว

Backend
Controller + Route Layer

จัดการ Logic ทั้งหมด

Database
Model Layer

เก็บข้อมูลและจัดการข้อมูล

รูปแบบการออกแบบที่ใช้

MVC Pattern
Frontend = View (แสดงผล)
Routes/Controllers = Controller (จัดการ Logic)
Database = Model (จัดการข้อมูล)

Layered Architecture
Route → Controller → Database
แต่ละชั้นแยกหน้าที่ชัด แก้ไขง่าย ไม่กระทบกัน

Middleware Pattern
มีตัวกลางตรวจสอบสิทธิ์ (Auth) และจัดการไฟล์ก่อนเข้า Controller
เพิ่มความปลอดภัย และทำให้โค้ดสะอาด

RESTful API
สื่อสารผ่าน API ตาม Resource เช่น /auth, /profile
Frontend ไม่เข้าถึง Database โดยตรง

Component-Based Frontend
แยก UI เป็นส่วนย่อย เช่น Navbar, Sidebar, หน้า Auth, หน้า Mainใช้ซ้ำได้ และจัดโครงสร้างหน้าเว็บชัดเจน

เครื่องมือที่ใช้

Front-end

Next.js

TypeScript

Tailwind CSS

Back-end

Express.js

TypeScript

Database

MySQL

Deploy

Vercel

Render

Railway

การพัฒนาและคุณภาพของโค้ด

โปรเจกต์นี้ใช้แนวทาง Agile โดยแบ่งการทำงานเป็น Sprint ละ 2 สัปดาห์ ใน Sprint แรกจะเน้นการพัฒนาฟังก์ชันหลัก เช่น การเข้าสู่ระบบและการจัดการคำร้อง ส่วน Sprint ที่สองจะเป็นการเพิ่มฟีเจอร์เสริม และเน้นการทดสอบระบบ เช่น Unit Test ซึ่งช่วยให้ระบบมีคุณภาพและสามารถปรับปรุงได้อย่างต่อเนื่อง

การแบ่งงาน

- ใช้ Agile แบ่งงานเป็น Sprint (2 สัปดาห์)
- ใช้ GitHub ในการจัดการงาน
- แยก branch (develop / main)
- commit และ merge โค้ด

คุณภาพโค้ด

- ใช้ TypeScript ลด error
- ใช้ Jest สำหรับ Unit Test

Testing&QA

```
PASS  src/tests/integration/api.integration.test.ts (11.105 s)
-------------------------|---------|----------|---------|---------|-------------------
File                     | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------------|---------|----------|---------|---------|-------------------
All files                |    93.3 |    84.69 |    93.1 |      97 |
 controllers             |   93.36 |    83.69 |   92.85 |   97.43 |
  authController.ts      |     100 |     87.5 |     100 |     100 | 16,32
  complaintController.ts |   92.57 |    82.89 |   91.66 |   97.12 | 158,347-349
 middleware              |    92.3 |      100 |     100 |    90.9 |
  authMiddleware.ts      |    92.3 |      100 |     100 |    90.9 | 22
-------------------------|---------|----------|---------|---------|-------------------

Test Suites: 7 passed, 7 total
Tests:       105 passed, 105 total
Snapshots:   0 total
Time:        12.665 s
Ran all test suites.
```

Security Assessment Summary

ใช้ Jest ในการทำ Unit และ Integration Testing โดยอ้างอิง Test Case จริง

Test Coverage

93.3% Statements

84.69% Branches

93.1% Functions

97% Lines

Quality Results

100%

Test ทั้งหมด 105 test ผ่านทั้งหมด เกิน 80% ทุกค่า

Test Suites: 7 passed

Tests: 105 passed

CI/CD & DevOps

01 Pipeline Overview

Developer push โค้ดขึ้น GitHub → Vercel และ Render รับสัญญาณ deploy อัตโนมัติทันที

02 Deployment Strategy

- Frontend (Next.js) --> Vercel --> Auto-deploy ทุกครั้งที่ push
- Backend (Express) --> Render --> Auto-deploy ทุกครั้งที่ push
- Database (MySQL) --> Railway --> Managed รันตลอดเวลา

03 Automation Benefits

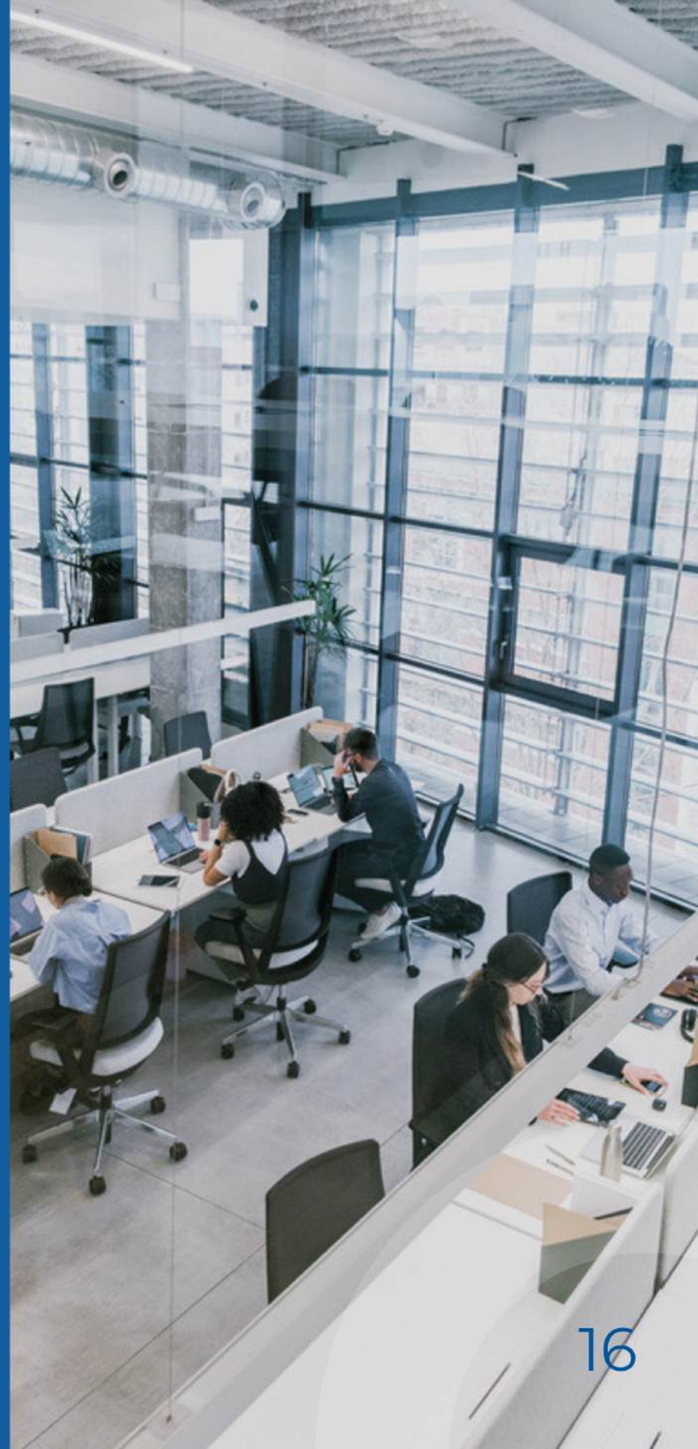
- ลดเวลา deploy จาก manual เป็น ~5 นาทีอัตโนมัติ
- ย้อนกลับ deploy เก่าได้ทุกเวอร์ชัน
- Environment Variables แยก local และ production ชัดเจน

สิ่งที่ทำได้ดี

- TypeScript ช่วยลด bug ตั้งแต่ตอนเขียนโค้ด
- Deploy ฟรีได้จริงด้วย Vercel + Render + Railway
- Git ช่วยให้ทีม 6 คนทำงานร่วมกันได้

ปัญหาที่พบระหว่างการพัฒนา และแนวทางแก้ไข

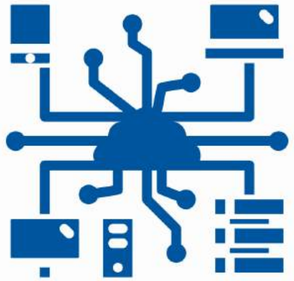
- Next.js SSR กับ localStorage → เพิ่มการเช็ค typeof window
- Render server sleep → แจ้งให้รอ warm up ก่อน demo
- Railway credit จำกัด → เปิด Serverless mode ประหยัด credit



สิ่งที่จะพัฒนาต่อ

เพิ่มการแจ้งเตือนผ่าน Email

พัฒนา Mobile Application



เพิ่ม WebSocket สำหรับ real-time

THANK YOU